

The fuel replenishment problem: A split-delivery multi-compartment vehicle routing problem with multiple trips

L. Wang^{a,b}, J. Kinable^{a,*}, T. van Woensel^a

^aSchool of Industrial Engineering, Eindhoven University of Technology, Eindhoven, The Netherlands

^bDepartment of Oil and Gas Storage and Transportation Engineering, China University of Petroleum, Beijing, China

ARTICLE INFO

Article history:

Received 25 May 2019

Revised 31 January 2020

Accepted 1 February 2020

Available online 8 February 2020

Keywords:

Fuel replenishment

Multi-compartment

Multi-trip

Split-delivery

Vehicle routing

Adaptive large neighborhood search

ABSTRACT

The Fuel Replenishment Problem (FRP) is a multi-compartment, multi-trip, split-delivery VRP in which tanker trucks transport different types of petrol, separated over multiple vehicle compartments, from an oil depot to petrol stations. Large customer demands often necessitate multiple deliveries. Throughout a single working day, a tanker truck returns several times to the oil depot to resupply. A solution to the FRP involves computing a delivery schedule of minimum duration, thereby determining for each vehicle (1) the allocation of oil products to vehicle compartments, (2) the delivery routes, and (3) the delivery patterns. To solve FRP efficiently, an Adaptive Large Neighborhood Search (ALNS) heuristic is constructed. The heuristic is evaluated on data from a Chinese petroleum transportation company and compared against exact results from a MILP model and lower bounds from a column generation approach. In addition, we perform sensitivity analysis on different problem features, including the number of vehicles, products, vehicle compartments and their capacities. Computational results show that the ALNS heuristic is capable of solving instances with up to 60 customers and 3 different products in less than 25 minutes with an average optimality gap of around 10%. On smaller instances, the heuristic finds optimal solutions in significantly less time than the exact MILP formulation.

© 2020 Elsevier Ltd. All rights reserved.

1. Introduction

This paper deals with a multi-compartment, multi-trip and split-delivery routing problem that occurs, for example, in the context of petrol station replenishment. We denote this problem as the Fuel Replenishment Problem (FRP). The cost of vehicle transportation accounts for 60% to 70% of the total cost of the existing refined oil logistics system (Lihua, 2012). Therefore, efficient vehicle routing is crucial. In general, a vehicle is separated into two to five compartments storing several incompatible oil products. These compartments are inflexible to avoid leakage and contamination. Using debit meters, the content in one compartment can be split among several customers. Likewise, a petrol station can be supplied by multiple vehicles (split-delivery).

Petrol stations are replenished by a fleet of heterogeneous vehicles. The vehicles load fuel at a central depot, and then travel to one or more petrol stations to deliver their fuel. Once empty, the vehicles return to the central depot to resupply after which they

continue servicing other fuel stations. At the end of the day, the vehicles return to the central depot.

Each fuel station carries one or more types of fuel. The fuel demand of some of the bigger fuel stations, typically located near busy highways, often exceeds the capacity of a single vehicle. Therefore, multiple vehicles may be required to satisfy the demand of a single station. The latter contrasts to traditional VRP problems in which a customer is visited only once by a single vehicle.

Following the categorization proposed by Coelho and Laporte (2015), the Multi-Compartment Delivery Problem considered in this paper can be classified as a split-split delivery problem. For split-split VRPs, three decisions need to be made: (1) the vehicle routes, (2) the compartment assignment, i.e. the capacity of each product, and, (3) the delivery pattern of each route and each product. Every vehicle is allowed to execute multiple trips.

The main scientific contributions of this paper are as follows:

- We propose a comprehensive variant of the split-delivery vehicle routing problem (SD-VRP) considering multiple compartments, multiple trips and a heterogeneous fleet of vehicles.
- We develop a Mixed Integer Programming (MILP) formulation for the described routing problem. To benchmark, we obtain exact solutions for some small instances using CPLEX. We addition-

* Corresponding author.

E-mail addresses: mxdsjyd@163.com (L. Wang), j.kinable@tue.nl (J. Kinable), t.v.woensel@tue.nl (T. van Woensel).

ally develop a column generation approach to compute lower bounds as an additional benchmark.

- We adapt the Adaptive Large Neighborhood Search (ALNS) heuristic to solve our problem for larger instances. The solutions obtained by the ALNS approach are compared to the solutions of both the MILP model and the lower bound obtained by the column generation approach. We show that our ALNS heuristic consistently outperforms MILP. The ALNS obtains the same or better solutions than the MILP for all small instances within 10 seconds per instance, whereas, for larger instances, solving the MILP directly does not give a feasible solution within a two hour time limit. For the largest instance with 60 customers and 20 vehicles, computation times of the ALNS stay well below 25 minutes, while obtaining an average optimality gap of less than 10%.

A problem similar to the FRP discussed in this paper is found in [Coelho and Laporte \(2015\)](#). [Coelho and Laporte \(2015\)](#) consider several exact branch-and-cut algorithms for a class of multi-compartment delivery problems based on formulations for multi-period inventory-routing problems. Using a branch-and-cut approach, they are able to solve instances with up to 10 customers, 3 products, 3 compartments, and 6 vehicles to optimality. In this work, the focus is on the design of an efficient metaheuristic capable of solving a FRP for a single planning period. In contrast to the VRPs discussed in [Coelho and Laporte \(2015\)](#), the tanker trucks in our problem typically perform multiple trips a day. Moreover, the goal is to compute schedules of minimum duration, thereby spreading the workload over the available vehicles, as opposed to merely minimizing the total travel time.

The remainder of this paper is organized as follows: [Section 2](#) presents a brief review of the existing work related to this paper. [Section 3](#) formally describes our problem on-hand and gives details on the mathematical formulation. [Section 4](#) introduces a column generation method, obtaining a lower bound for our problem on-hand. In [Section 5](#), we develop an adaptive large neighborhood search (ALNS). In [Section 6](#), we report on the computational results. We conclude the paper in [Section 7](#).

2. Related literature

In this paper, we discuss a vehicle routing problem involving multiple compartments, split-delivery, and multiple trips. There are a few papers related to the problem discussed in this paper. These papers are based on the assumption that the compartments in the vehicle are not equipped with debit meters, implying that a compartment should be completely emptied once the

delivery is started. [Avella et al. \(2004\)](#) studied a petrol replenishment problem involving product packing and vehicle routing. The authors simplified the problem by an assumption that each compartment of the vehicle must travel either full or empty. Other authors discussed variants of this petrol replenishment problem: single period ([Cornillier et al., 2008a](#)), multi-period ([Cornillier et al., 2008b](#)), with time windows ([Cornillier et al., 2009](#)) and multi-depot ([Cornillier et al., 2012](#)). In these papers, the authors limit the number of customers (up to 4) for each trip according to a distribution policy used in practice.

[Popović et al. \(2012\)](#) propose a Variable Neighborhood Search(VNS) heuristic for a multi-period Inventory Routing Problem (IRP) involving the distribution of petrol with a multi-compartment vehicle fleet. Similar to the aforementioned papers, [Popović et al. \(2012\)](#) also assume only full compartments are delivered, and each vehicle can visit up to 3 customers per trip. [Vidović et al. \(2014\)](#) developed a Variable Neighborhood Search (VNS) heuristic for a multi-product multi-period IRP with a multi-compartment homogeneous fleet. In addition, they proposed a MILP model for the problem and limited each route to 4 customers. [Benantar et al. \(2016\)](#) considered a multi-compartment Inventory Routing Problem (IRP) with time windows. In contrast to [Cornillier et al. \(2009\)](#), they do not limit the number of customers visited in a route.

[Table 1](#) presents a comparison of the main references after 2000.

The split-delivery vehicle routing problem (SDVRP) also received attention in the literature. [Dror and Trudeau \(1989\)](#) and [Dror and Trudeau \(1990\)](#) proposed a local search approach to solve the SDVRP. Afterwards, more complex approaches, such as metaheuristics and hybrid algorithms were introduced. [Sierksma and Tijssen \(1998\)](#) proposed an algorithm based on a column generation algorithm, combined with other heuristics to solve a flight scheduling problem. [Archetti et al. \(2006\)](#) proposed a Tabu search algorithm for the SDVRP. These authors also introduced a hybrid Tabu search and integer programming algorithm. There are few papers that studied exact algorithms for SDVRP. [Lee et al. \(2006\)](#) developed a dynamic programming method with infinite state and action spaces. [Jin et al. \(2007\)](#) proposed a two-stage algorithm with valid inequalities to solve the SDVRP to optimality. [Feillet et al. \(2006\)](#) introduced a branch-price-cut method to solve the SDVRP. Another branch-price-cut algorithm capable of solving SDVRP instances up to 100 customers is presented by [Desaulniers \(2010\)](#). [Archetti et al. \(2011\)](#) enhanced the latter work by introducing several valid inequalities, and by proposing an efficient Tabu search to solve the pricing subproblem. Finally, a vari-

Table 1
Literature comparison.

	Fleet	TW	MC	MP/MT	IRP	MD	# Cust	Split
Avella et al. (2004)	Heterog.	no	no	-	no	no	unlim.	-
Abdelaziz et al. (2002)	Heterog.	no	yes	-	no	no	unlim.	u-u
Ng et al. (2008)	Heterog.	no	yes	-	no	no	unlim.	u-u
Vidović et al. (2014)	Homog.	no	yes	MP	yes	no	max. 3	u-u
Popović et al. (2012)	Homog.	no	yes	MP	yes	no	max. 4	u-u
Benantar et al. (2016)	Heterog.	yes	yes	-	yes	no	unlim.	u-u
Cornillier et al. (2008a)	Heterog.	no	yes	-	yes	no	max. 2	u-u
Cornillier et al. (2008b)	Heterog.	no	yes	MP	yes	no	max. 2	u-u
Cornillier et al. (2009)	Heterog.	yes	yes	MT	yes	no	max. 4	u-u
Cornillier et al. (2012)	Heterog.	yes	yes	MT	yes	yes	max. 3	u-u
Coelho and Laporte (2015)	Homog.	no	yes	MP	yes	no	unlim.	u-u, u-s, s-u, s-s
This paper	Heterog.	no	yes	MT	no	no	unlim.	s-s

*TW: Time Window; MC: Multi-Compartment; MP: Multi-Period; MT: Multi Trip; IRP: Inventory Routing Problem; MD: Multi-Depot; # Cust: Maximum number of customers per route; Split: Possibility to split the compartment and the demand of the customer or not ([Coelho and Laporte \(2015\)](#) for the details), 'u' means unsplit and 's' means split.

ant of the SDVRP with time-windows, split delivery, and weight-related cost is addressed by Luo et al. (2016).

The literature on multi-compartment vehicle routing problem (MCVRP) covers various application domains including waste collection (Muyldermans and Pang (2010) and Henke et al. (2015)), foods or groceries delivery (Derigs et al., 2011; Hübner and Ostermeier, 2018) and in livestock transportation (Oppen and Løkketangen, 2008). Lahyani et al. (2015) solve a problem involving the collection of olive oil with different qualities with a branch-and-cut algorithm. El Fallahi et al. (2008) augment classical VRP instances with vehicle compartments. Mendoza et al. (2010) use a memetic algorithm to solve an MCVRP with stochastic demands and developed a constructive heuristics for their algorithm (Mendoza et al., 2011). Melechovsky (2013) extend the MCVRP by considering time windows and solved the problem using a VNS algorithm.

3. Problem definition and mathematical model

The FRP is formally defined on a complete, undirected graph $G = (V, E)$ with node set V , and edge set $E = \{(i, j) : i, j \in V, i < j\}$. The node set $V = \{0, 1, 2, \dots, n\}$ consists of a supply depot 0, and a set of $1, 2, \dots, n$ customers (e.g. the petrol stations). Each customer $i \in V$ has a non-negative demand $d_{i,p}$ for some (oil) product $p \in P$. For brevity, we often denote the demand $d_i \in \mathbb{R}^{|P|}$ of some customer $i \in V$ as a vector, i.e. $d_i = (d_{i,1}, d_{i,2}, \dots, d_{i,|P|})$. Assume that the depot has no demand ($d_0 = \mathbf{0}$). Servicing is performed by a heterogeneous fleet of vehicles K which are stationed at the supply depot. Each vehicle $k \in K$ has C^k compartments of capacity Q^k . For simplicity, we assume that the compartments in a vehicle are of equal size. Between any pair of locations $i, j \in V$, a positive travel time $d_{ij} > 0$ is specified (satisfying the triangle inequality). A summary of notation used is provided in Table 2.

A solution to FRP consists of a set of itineraries, one for each vehicle, which minimizes both makespan and total travel time, while collectively satisfying the demand of all customers. More precisely, a vehicle itinerary describes the order in which a vehicle must visit its customers, the quantity of each product it must load when the vehicle (re-)visits the supply depot, and the quantity of each type of fuel it must deliver during each of the customer visits. We will refer to a product delivery as a customer *visit*. Similarly, a *trip* is defined as the sequence of visits performed by a vehicle between its departure from the supply depot and its next return to the supply depot. Note that, since a vehicle is able to resupply, a vehicle can re-visit the same customer during different trips.

Let T_k be the set of trips performed by vehicle $k \in K$. Trips in T_k can be performed in arbitrary order, since each trip starts and ends at the same depot, and there are no temporal constraints imposed on the customer visits. Each trip $t \in T_k$ constitutes an ordered sequence of customer visits. Each visit v is uniquely associated with a customer $i_v \in V$. A vector $qt_v = (qt_{v1}, qt_{v2}, \dots, qt_{v|P|})$ specifies the

quantity of each product delivered to customer i_v during visit v . The set of trips T is defined as $\bigcup_{k \in K} T_k$.

In the remainder of this paper, we denote by $\delta(S)$, for any subset of vertices $S \subseteq V$, the set of edges with exactly one endpoint in S . When S is a singleton vertex, we use the shorthand $\delta(i)$ instead of $\delta(\{i\})$. The set $E(S)$ denotes the set of edges with both endpoints in S .

We formulate the FRP as a Mixed Integer Linear Program (MILP). For each vehicle $k \in K$, the MILP explicitly models a number of trips $T_k = \{0, 1, 2, \dots, T_k^{\max}\}$, where $T_k^{\max}$ is an upper bound on the number of trips vehicle k should make. Here $T_k^{\max}$ can be computed by assuming that all deliveries are performed by a single vehicle k , i.e.:

$$T_k^{\max} = \left\lceil \frac{\sum_{i \in V} \sum_{p \in P} \left\lceil \frac{q_{ip}}{Q^k} \right\rceil}{C^k} \right\rceil \quad (1)$$

In the MILP formulation below, binary variables x_{ij}^t model whether vehicle $k \in K$ traverses edge $(i, j) \in E$ during trip $t \in T_k$. Similarly, continuous variables u_i^{tp} record the amount of product $p \in P$ delivered during trip t to customer $i \in V$. The number of vehicle compartments dedicated to a product $p \in P$ during a trip t is modeled by integer variables z^{tp} . Finally, binary variables y_i^t are used to identify whether customer i is visited during trip t .

The FRP model is formulated as follows:

$$\text{minimize } \alpha \tau + \beta \sum_{k \in K} \sum_{t \in T_k} \sum_{(i,j) \in E} d_{ij} x_{ij}^t \quad (2)$$

$$\text{s.t. } \tau \geq \sum_{t \in T_k} \sum_{(i,j) \in E} d_{ij} x_{ij}^t \quad \forall k \in K \quad (3)$$

$$\sum_{j \in V \setminus \{0\}} x_{0j}^t \leq 1 \quad \forall k \in K, t \in T_k \quad (4)$$

$$\sum_{j \in V, i < j} x_{ij}^t + \sum_{j \in V, j < i} x_{ji}^t = 2y_i^t \quad \forall k \in K, t \in T_k, i \in V \setminus \{0\} \quad (5)$$

$$\sum_{(i,j) \in E(S)} x_{ij}^t \leq |S| - 1 \quad \forall S \subseteq N \setminus \{0\}, |S| \geq 2, k \in K, t \in T_k \quad (6)$$

$$\sum_{k \in K} \sum_{t \in T_k} u_i^{tp} = q_{ip} \quad \forall i \in V, p \in P \quad (7)$$

$$\sum_{i \in V \setminus \{0\}} u_i^{tp} \leq z^{tp} Q^k \quad \forall k \in K, t \in T_k, p \in P \quad (8)$$

$$\sum_{p \in P} z^{tp} \leq C^k \quad \forall k \in K, t \in T_k \quad (9)$$

$$u_i^{tp} \leq y_i^t Q^k C^k \quad \forall k \in K, t \in T_k, p \in P, i \in V \setminus \{0\} \quad (10)$$

$$0 \leq z^{tp} \leq C^k \quad \forall k \in K, t \in T_k, p \in P \quad (11)$$

$$0 \leq u_i^{tp} \leq \min\{Q^k C^k, q_{ip}\} \quad \forall i \in V, k \in K, t \in T_k, p \in P \quad (12)$$

$$x_{ij}^t \in \{0, 1\} \quad \forall (i, j) \in E, k \in K, t \in T_k \quad (13)$$

$$y_i^t \in \{0, 1\} \quad \forall i \in V \setminus \{0\}, k \in K, t \in T_k \quad (14)$$

Using non-negative scalars α and β , the weighted objective function (2) minimizes the makespan as well as the total travel time. Constraints (3) track the latest completion time (makespan) of the vehicle itineraries. Constraints (4)–(6) implement the trip

Table 2
Notation.

Parameter/Set	Description
$V = \{0, 1, 2, \dots, n\}$	Set of stations and depot
$E = \{(i, j) i, j \in V, i < j\}$	Set of undirected edges
K	Set of vehicles
P	Set of different products
q_{ip}	Demand of station $i \in V$ for product $p \in P$
Q^k	Capacity of a single compartment of vehicle $k \in K$
d_{ij}	Travel time associated with arc $(i, j) \in E$
C^k	Number of compartment in vehicle $k \in K$
T_k	Trips performed by vehicle k
T	Set of all trips: $\bigcup_{k \in K} T_k$

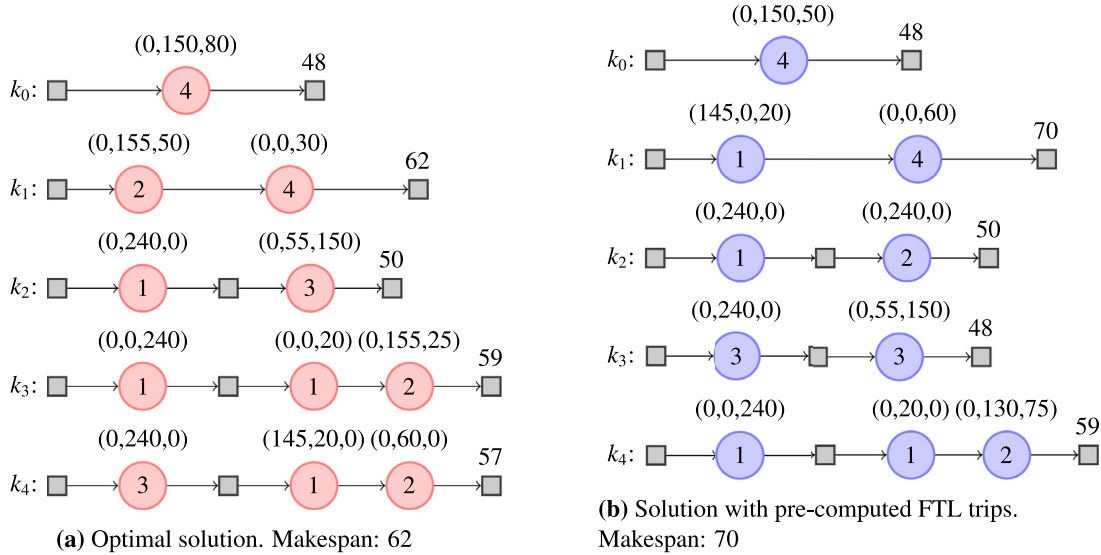


Fig. 1. Solutions to the FRP instance given in Table 3. A square represents the depot, a circle represents a visit v to the customer indicated inside the circle. The vector above each circle specifies the delivery quantity per product (q_{tv}). The number above the final depot in each route represents the route duration.

structure. A vehicle can depart from the depot at most once during each trip (Constraints (4)). Constraints (5),(6) respectively implement flow preservation and eliminate subtours. Next, Constraints (7)–(9) deal with the vehicle capacities and customer demands. The demand for each product for each customer must be satisfied (Constraint (7)). Each vehicle compartment can be dedicated to at most one product (Constraint (9)), but product allocations may change between trips. The maximum amount of fuel a vehicle can deliver during a single trip is bounded by the number of compartments dedicated to this type of fuel, multiplied by the compartment capacity (Constraint (8)). From Constraints (8), (9), it follows that the maximum amount of product provided by vehicle k during a single trip cannot exceed $Q^k c^k$. Finally, Constraints (10) link the u_i^{tp} and y_j^t variables.

To reduce symmetry in the model, the following constraints are added:

$$\sum_{i \in V} \sum_{\substack{j \in V, \\ i < j}} d_{ij} x_{ij}^t \geq \sum_{i \in V} \sum_{\substack{j \in V, \\ i < j}} d_{ij} x_{ij}^{t+1} \quad \forall k \in K, t \in T_k \setminus T_k^{\max} \quad (15)$$

These constraints order the trips performed by each vehicle, thereby ensuring that the duration of each consecutive trip is not longer than its preceding trip.

The FRP, as defined by Constraints (2)–(15), belongs to the class of NP-hard problems (reduction to the Capacitated Vehicle Routing Problem). An example problem instance of FRP having 4 customers, 3 products and five 3-compartment vehicles is given in Table 3.

Table 3
Instance data for $V = \{0, 1, 2, 3, 4\}$.

(a) Demands of the customers					
	p_1	p_2	p_3		
1	145	260	260		
2	0	370	75		
3	0	295	150		
4	0	150	110		
(b) Distance Matrix					
	0	1	2	3	4
0	0	13	12	12	24
1	13	0	8	23	33
2	12	8	0	18	26
3	12	23	18	0	12
4	24	33	26	12	0

Each vehicle compartment has a capacity of 80 units. An optimal solution to this problem instance in which the makespan component is prioritized over the total travel distance is shown in Fig. 1a. Observe that each vehicle has a maximum capacity of $80 \times 3 = 240$ units, and that some customers require more than 240 units of products. It might be tempting to create dedicated Full Truck Load (FTL) trips for customers whose demand exceeds the capacity of a single vehicle, but this approach leads to suboptimal solutions. As an example, we could create an FTL trip for customer 2 in which a vehicle delivers 240 units of p_2 . In the resulting optimization problem we would have to assign the FTL trips to vehicles, as well as any additional trips required to serve the remaining demand of each customer. The resulting schedule is depicted in Fig. 1b: despite a higher number of FTL trips, the makespan of the solution 1b is longer than the makespan of solution 1a.

4. Lower bounds

The model presented in Section 3 explicitly models trips for each vehicle. As a consequence, the performance of this model is strongly correlated to the number of trips allocated to each vehicle. The bound $T_k^{\max}$ computed in Eq. (1) is typically weak. Computing stronger bounds is non-trivial. Consequently, the scalability of the MILP model is limited.

In order to assess the quality of the ALNS heuristic presented in Section 5 on larger problem instances, we propose an alternative mathematical model that can be used to compute strong bounds on the optimal objective value.

For a given vehicle $k \in K$, let T_k be the set of all potential resource feasible trips vehicle k can make. Each trip must visit one or more customers, and start and end at the supply depot. Moreover, let δ_t denote the duration of trip t , μ_{ip}^t the amount of product $p \in P$ delivered to customer $i \in V$ during trip t , and let binary parameter e_{ij}^t indicate whether edge $(i, j) \in E$ is traversed in trip t . Using binary assignment variables w_k^t which assign trips to vehicles, we then formulate the FRP as follows:

MP :

$$\text{minimize } \alpha \tau + \beta \sum_{t \in T} \sum_{k \in K} \delta_t w_k^t \quad (16)$$

$$\text{s.t. } \tau \geq \sum_{t \in T} \delta_t w_k^t \quad \forall k \in K \quad (17)$$

$$\sum_{t \in T} \sum_{k \in K} \mu_{ip}^t w_k^t \geq q_{ip} \quad \forall i \in V, p \in P \quad (18)$$

$$w_k^t \in \mathbb{Z}_0 \quad \forall t \in T, k \in K \quad (19)$$

$$\tau \geq 0 \quad (20)$$

The weighted objective function (16), together with Constraint (17) minimize the makespan plus the total travel time. Constraints (18) ensure that the demand of each customer is satisfied.

Due to the large number of potential trips for each vehicle, this model cannot be solved directly by an ILP solver. Instead a Column Generation (CG) procedure is developed. Let LPM be the Linear Program Relaxation obtained from model MP by relaxing the integrality conditions on the w_k^t variables (Eq. (19)). Associate dual variables $\zeta_k \leq 0$, $\eta^{ip} \geq 0$ with constraints (17), (18) respectively. The dual problem DMP of LPM then becomes:

DMP :

$$\text{maximize } \sum_{i \in V} \sum_{p \in P} \eta^{ip} q_{ip} \quad (21)$$

$$\text{s.t. } \delta_t(\zeta^k - \alpha) + \sum_{i \in V} \sum_{p \in P} \mu_{ip}^t \eta^{ip} \leq 0 \quad \forall t \in T, k \in K \quad (22)$$

$$-\sum_{k \in K} \zeta^k \leq \alpha \quad (23)$$

$$\zeta^k \leq 0 \quad \forall k \in K \quad (24)$$

$$\eta^{ip} \geq 0 \quad \forall i \in V, p \in P \quad (25)$$

Following a standard CG procedure (e.g., see Desaulniers, 2010) LPM is solved for a small subset of trips $T'_k \subset T_k$ for all vehicles $k \in K$. At every iteration of the CG procedure, a pricing problem is solved for each vehicle $k \in K$ to identify columns with negative reduced cost, i.e. columns for which $\delta_t(\zeta^k - \alpha) + \sum_{i \in V} \sum_{p \in P} \mu_{ip}^t \eta^{ip} > 0$, which are then added to the set T'_k . A simplified version of the MILP model in Section 3 can be used to solve the pricing problem for a given vehicle $k \in K$; the definitions of the variables remain the same, but their t index is dropped. The pricing problem then becomes:

PRICING(k) :

$$\text{maximize } (\zeta^k - \alpha) \sum_{(i,j) \in E} d_{ij} x_{ij} + \sum_{i \in V \setminus \{0\}} \sum_{p \in P} \eta^{ip} u_{ip} \quad (26)$$

$$\text{s.t. Eq. (4) - (6), (8) - (10)} \quad (27)$$

When solving the Master Problem for a homogeneous set of vehicles as opposed to a heterogeneous set, it suffices to solve the pricing problem only once for a single vehicle k' during each iteration of the column generation algorithm, where $k' = \arg\max_{k \in K} \zeta^k$.

This observation follows directly from the definition of the pricing problem and the domains of the dual variables.

The master problem is strengthened by adding the following Capacity Inequalities:

$$\sum_{k \in K} \sum_{t \in T} \sum_{(i,j) \in \delta(S)} e_{ij}^t w_k^t \geq 2 \left\lceil \frac{\sum_{p \in P} \left\lceil \frac{d_p(S)}{Q} \right\rceil}{C} \right\rceil \quad \forall S \subseteq V \setminus \{0\}, |S| \geq 2 \quad (28)$$

Associating dual variables λ_S with inequalities (28) leads to the following adjusted pricing problem:

$$\delta_t(\zeta^k - \alpha) + \sum_{i \in V} \sum_{p \in P} \mu_{ip}^t \eta^{ip} + \sum_{S \subseteq V \setminus \{0\}, |S| \geq 2} \sum_{(i,j) \in \delta(S)} e_{ij}^t \lambda_S > 0 \quad (29)$$

This change to the pricing problem is easily accommodated for in the objective function (26) of our pricing problem formulation. Since the set of Capacity Inequalities is exponentially large, it is undesirable to add them all at once to the master problem. Instead, a separation procedure is used to separate violated inequalities. We first run the CG procedure until no more columns with negative reduced cost can be identified. Next the separation procedure is invoked. Whenever violated inequalities are identified, they are added to MP and the CG procedure is resumed. To separate Capacity Inequalities, we first invoke the heuristic route-based algorithm proposed by Archetti et al. (2011) (see Section 4.2 in Archetti et al. (2011)). If this method fails to identify violated inequalities, we subsequently invoke the computationally more expensive partial enumeration routine by Desaulniers (2010) (see 5.2.1 in Desaulniers (2010)).

Finally, when no more columns with negative reduced cost or violated inequalities can be identified, an optimal solution to LPM is obtained. Note that, due the relaxation of the integrality conditions of the w_k^t variables, the resulting solution can be fractional. The CG procedure can be embedded in a branch-and-price framework in order to guarantee integer feasible solutions. The latter is however outside the scope of this paper. We refer interested readers to Desaulniers (2010), Archetti et al. (2011), and Luo et al. (2016). The optimal solution to LPM constitutes a valid lower bound on the optimal objective value of our FRP.

5. ALNS heuristic

This section presents an Adaptive Large Neighborhood Search (ALNS) heuristic for the FRP, based on the ALNS framework outlined in Ropke and Pisinger (2006) and Pisinger and Ropke (2007). Similar to a conventional Large Neighborhood Search (LNS) heuristics (Shaw, 1998), ALNS follows a ruin-and-recreate paradigm. However, in contrast to LNS, ALNS relies on multiple destroy and repair operators, which are randomly selected through preferential selection. An overview of our ALNS framework is provided in Algorithm 1. During each iteration of the heuristic, a destroy operator is chosen to ruin the current solution, thereby removing visits, trips, or even entire routes from the schedule. Next, a repair operator is selected to repair the (partially) destroyed solution. The destroy/repair procedure is embedded in a local search framework, in our case Simulated Annealing, to determine whether the new solution is accepted as the starting point of the next iteration, or rejected in favor of the old solution. The ALNS heuristic terminates when either a time limit is reached, or a maximum number of non-improving moves have been performed.

The ALNS heuristic is initialized with a feasible solution obtained through a constructive heuristic (Section 5.1). At each iteration of the ALNS heuristic, a destroy and a repair operator are selected based on their past performance. A description of this selection procedure is provided in Section 5.2, followed by an overview of destroy and repair operators (Section 5.3). Each invocation of a destroy operator removes one or more visits from the schedule and adds them to a pool of removed visits. Notice that multiple visits belonging to the same customer can be removed during a single invocation. These visits are added separately to the pool of removed visits; we do not aggregate these visits into a larger visit. After the destroy operator removed a number of visits, a repair operator is selected to re-insert the visits stored in the removal pool back into the schedule. Typically there exist multiple feasible alternatives to re-insert a visit back into the schedule, each leading to a different solution with a different objective value. The repair operator ultimately decides which of these alternatives is selected.

When inserting a visit somewhere in the schedule, we ensure that the resulting (partial) schedule remains feasible. This implies that a visit can only be inserted into an existing trip if this does

Algorithm 1: ALNS with SA.

Input: Destroy operators Δ , repair operators Ψ , initial temperature T , cooling rate κ , initial solution X_{init}

Output: A feasible solution X_{best}

```

1  $X_{current} \leftarrow X_{best} \leftarrow X_{init}$ 
2  $it_{nonImp} \leftarrow 0$ 
3 repeat
4   Select a destroy operator  $\delta \in \Delta$  with probability
      $w_\delta / \sum_{i=1}^{|\Delta|} w_i$ 
5   Select a repair operator  $\psi \in \Psi$  with probability
      $w_\psi / \sum_{i=1}^{|\Psi|} w_i$ 
6    $X_{new} \leftarrow \psi(\delta(X_{current}))$ 
7   if  $obj(X_{new}) < obj(X_{current})$  then
8      $it_{nonImp} \leftarrow 0$ 
9      $X_{current} \leftarrow X_{new}$ 
10    if  $obj(X_{current}) < obj(X_{best})$  then
11       $X_{best} \leftarrow X_{current}$ 
12  else
13     $it_{nonImp} \leftarrow it_{nonImp} + 1$ 
14    Generate a random number  $\epsilon \in [0, 1]$ 
15    if  $\epsilon < e^{-(obj(X_{new}) - obj(X_{current}))/T}$  then
16       $X_{current} \leftarrow X_{new}$ 
17   $T \leftarrow \kappa T$ 
18  Update operator probabilities
19 until  $timeLimit$  or  $it_{nonImp} \geq nonImp_{max}$ 

```

not exceed the capacity of the vehicle performing the trip. Whenever a visit is inserted into a trip which already contains a visit for the same customer, we simply merge these visits and the respective delivered quantities.

We consider three strategies to insert an individual visit into the schedule: Integral Insertion (II), Split Trip Insertion (STI), and Split Insertion (SI).

The II strategy simply inserts a visit into an existing trip. The STI strategy splits an existing trip t into two trips t_1, t_2 , and assign the visit to either one of the two trips. Splitting a trip is performed by inserting a depot visit in between the sequence of customer visits. If this depot visit is inserted after the last customer visit, then this is equivalent to creating a new empty trip. To limit the number of alternative solutions to be evaluated, we require that the visit that has to be inserted becomes either the last visit in trip t_1 , or the first visit in t_2 . Finally, in contrast to the II strategy, the SI strategy does not reinsert a visit v back into the schedule; instead a number of smaller visits are inserted such that the amounts of each product supplied by the smaller visits add up to qt_v . In doing so, the SI strategy aims to increase vehicle utilization by claiming left-over capacity of vehicles. In our implementation of the SI strategy, we do not exhaustively consider every possible alternative to split a visit into smaller visits which can be distributed over the existing trips, as the number of alternatives is typically too large. Instead, we devised a fast, heuristic procedure to compute SIs. Note that this approach does not guarantee to identify any feasible SI even if one exists.

An overview of the SI strategy is given in Algorithm 2. Let $c_i(t)$ be the minimum cost incurred when inserting a visit for customer i into trip $t \in T$, where $c_i(t) = 0$ when trip t already visits customer i . When splitting a visit v into multiple smaller visits, we must ensure that the product quantities delivered in the resulting visits add up to qt_v . An iterative procedure, described in Algorithm 2, is used to split a visit v for customer i_v . At each iteration, the algorithm determines the best trip t to insert a new visit

Algorithm 2: Split insertion - splits a visit into multiple smaller visits.

Input: visit v for some customer $i_v \in V$

```

1  $cost \leftarrow 0$   $\triangleright$  total cost incurred by splitting visit  $v$ 
2  $qt \leftarrow qt_v$ 
3  $T^s = \{t \in T : 0 < a(t, qt) < qt\}$ 
4 while  $qt > 0$  and  $T^s \neq \emptyset$  do
5    $t' = \operatorname{argmin}_{t \in T^s} \frac{c_{i_v}(t)}{|a(t, qt)|}$   $\triangleright$  trip with lowest weighted
     insertion cost
6    $cost \leftarrow cost + c_{i_v}(t')$ 
7   insert  $(v', t')$  and set  $qt_{v'} = a(t', qt)$ ,  $i_{v'} = i_v$ 
8    $qt \leftarrow qt - qt_{v'}$ 
9    $T^s = \{t \in T^s : 0 < a(t, qt) \leq qt\}$ 
10 if  $qt > 0$  then
11   fail()

```

v' for customer i_v , based on a ratio between the insertion cost and the product quantities that can be delivered in trip t . This process is repeated until visits equivalent to a demand of qt_v are inserted, or the set of trips with residual capacity is exhausted. In the latter case, the operator fails to split delivery v . A vector qt , initialized to qt_v , is used to track the remaining demand for each product after each iteration of the algorithm.

Note that to prevent too much fragmentation, the SI operator is only allowed to insert visits into trips which cannot be used by the II operator (trips that do not have enough residual capacity to serve qt_v entirely). The function $a(t, qt) : (T, \mathbb{R}^{|P|}) \rightarrow \mathbb{R}^{|P|}$ is used to determine the quantity $st \in \mathbb{R}^{|P|}$, $0 \leq st \leq qt$, of products that can be delivered to customer i_v in trip t , based on the residual capacity of the vehicle performing trip t . A precise definition of the function $a(t, qt)$ is provided in Algorithm 3. The function first attempts to fill up any partially filled compartments of the vehicle serving trip t . Next, a greedy procedure is used to fill up any unused compartments of the same vehicle. Preference is given to products for

Algorithm 3: Implementation of $a(\tau, qt)$.

Input: demand vector $qt \in \mathbb{R}^{|P|}$, trip $\tau \in T$

Output: vector $sv \in \mathbb{R}^{|P|}$, $0 \leq sv \leq qt$: quantity of products that would be supplied to customer i_v if a new visit v' for customer i_v were to be inserted into trip τ

```

 $\triangleright$  Fill up partially filled compartments. Let  $res_{tp}$  be
the total unused capacity of non-empty compartments
holding product  $p \in P$  of the vehicle serving trip  $t \in T$ .
1  $sv = [\min\{res_{tp}, qt_p\}]_{p \in P}$ 
2  $\overline{qt} \leftarrow qt - sv$ 

 $\triangleright$  Allocate products to empty compartments. Let  $c_\tau$  be
the number of empty compartments of the vehicle serving
trip  $\tau$ .
3  $c' \leftarrow c_\tau$ 
4 while  $c' > 0$  do
5    $p' = \operatorname{argmax}_{p \in P} \overline{qt}_p - \sum_{t \in T \setminus \tau} res_{tp}$ 
6   if  $\overline{qt}_{p'} > 0$  then
7      $sv_{p'} = sv_{p'} + \min\{Q^k, \overline{qt}_{p'}\}$ 
8      $\overline{qt}_{p'} = \min\{0, \overline{qt}_{p'} - Q^k\}$ 
9      $c' \leftarrow c' - 1$ 
10 return  $sv$ 

```

Table 4
Adaptive adjustment of the operator scores.

Reward	Value	Eligibility criteria
σ_1	5	The last ALNS iteration produced a new best solution
σ_2	3	The last ALNS iteration produced a solution that was better than the solution from the previous iteration
σ_3	1	The solution from the last ALNS iteration is worse than the solution from the previous iteration, but was nevertheless accepted

which there is a large demand and for which there is little spare capacity in any of the other trips (line 5).

5.1. Initial solution

An initial feasible solution X_{init} for the ALNS heuristic is obtained through a simple constructive procedure. This procedure works in two separate phases:

1. Generate a set of vehicle-independent trips T such that the trips collectively satisfy the demand of all customers.
2. Assign each trip $t \in T$ to a vehicle.

When vehicles are heterogeneous, we only generate trips which meet the capacity requirements of the smallest vehicle $k' = \argmin_{k \in K} Q^k C^k$, having $C = C^{k'}$ compartments and $Q = Q^{k'}$ capacity per compartment.

The process of generating trips is subdivided into three steps, which attempt to generate trips with high vehicle utilization. The heuristic first generates dedicated trips for each customer $i \in V$ independently, which utilize the maximum capacity ($C \times Q$) of the smallest vehicle. Clearly, for a given customer $i \in V$, there exist $\left\lfloor \frac{\sum_{p \in P} q_{ip}/Q}{C} \right\rfloor$ such trips.

Let $\bar{q}_i$, $i \in V$, be a vector representing for each product the remaining, unsatisfied demand of a customer. After generating the Full Truck Load (FTL) trips, for a given customer $i \in V$, it follows that: $\sum_{p \in P} \left\lfloor \frac{\bar{q}_{ip}}{Q} \right\rfloor < C$, i.e. the residual demand of the remaining products is not sufficient to fill up another vehicle with C compartments of size Q . Next, a number of dedicated, Less-than-Truck-Load (LTL) trips for each customer are derived. We require that for each dedicated LTL trip, each vehicle compartment must hold some product, and that precedence is given to products for which the customer has the largest residual demand. As an example, let $C = 3$, $Q = 80$, and $d_i = (100, 20, 50)$ for some customer i . We create a dedicated LTL trip for customer i containing a single visit v with $qt_v = (100, 0, 50)$. The residual demand of customer i becomes $\bar{q}_i = (0, 20, 0)$. Again, for a single customer $i \in V$, there exist $\left\lfloor \frac{\sum_{p \in P} q_{ip}/Q}{C} \right\rfloor$ such LTL trips.

After updating $\bar{q}_i$ with the LTL trips for each customer $i \in V$, the following conditions hold: $|\{p \in P : \bar{q}_{ip} > 0\}| < C$ and $\sum_{p \in P} \bar{q}_{ip} < CQ$, i.e. the residual demand of customer i can be satisfied by a single vehicle. In the third, and last step of the trip generation phase, for each of the customers $i \in V$ with residual demand, we create a single visit v having $qt_v = \bar{q}_i$. Next, these visits are greedily inserted into the existing trips, following the insertion strategies outlined in the previous section, thereby always selecting the strategy that increases the objective the least.

Once all trips are generated, we assign them to vehicles. This assignment problem is solved by a greedy heuristic which allocates trips one by one, to the vehicle with the shortest itinerary, with ties broken arbitrarily. In case the vehicles are heterogeneous, a trip is only assigned to vehicle if that vehicle meets the capacity requirements to perform that trip. For instance, there might exist a vehicle which has more capacity than the smallest vehicle ($C \times Q$), but has fewer compartments. As such, this larger vehicle is unable to perform trips in which C different products have to be delivered.

5.2. Operator selection

As is conventional in ALNS, the destroy (resp. repair) operators are selected with a probability proportional to their past performance. Let Γ be the set of available operators (e.g. destroy operators), and w_γ a weight associated with each operator $\gamma \in \Gamma$. The probability that an operator $\gamma \in \Gamma$ is selected is set equal to $w_\gamma / \sum_{i \in \Gamma} w_i$.

The ALNS search is partitioned into sequences of 100 consecutive iterations. Throughout a sequence, the performance of each operator is monitored, and, at the end of the sequence the weights of the operators are adjusted according to their performance. Each time an operator is invoked during an ALNS iteration, it is able to collect a reward; a summary of potential rewards and their eligibility criteria is given in Table 4. Let π_γ be the total reward accumulated by operator γ during a sequence, and β_γ the number of times the operator was invoked during the sequence. At the end of each sequence, the normalized performance of an operator is given by $\frac{\pi_\gamma}{\beta_\gamma}$, and a new weight w'_γ for each operator $\gamma \in \Gamma$ is calculated as follows:

$$w'_\gamma = w_\gamma(1 - \xi) + \xi \frac{\pi_\gamma}{\beta_\gamma}, \quad (30)$$

where scalar ξ ($0 \leq \xi \leq 1$) controls how sensitive the weights react to changes in the operator's performance. In our implementation, at the beginning of the ALNS search, all operators have equal weight. Moreover, destroy and repair operators are selected independently from each other.

5.3. Destroy and repair operators

Six removal operators are used, the first three of them are directly adopted from Shaw (1998), Ropke and Pisinger (2006), Pisinger and Ropke (2007); the remaining three operators are tailored to our problem. To be self-contained, we include a brief description of each operator. For more details, we refer to the aforementioned references.

- Random Removal (RR): randomly remove $\phi \in [1, \rho|V|]$ visits from the schedule, where the scalar ρ is the destroy rate.
- Worst Removal (WR): Iteratively remove $\phi \in [1, \rho|V|]$ visits. Each removal yields a new schedule. In every iteration, we remove the visit whose removal from the schedule will benefit the objective value the most. Ties are broken arbitrarily.
- Shaw Removal (SR): Remove similar visits, where the similarity between a pair of visits u, v , $u \neq v$, is expressed by a similarity metric $s(u, v)$. Let $l_{uv} = -1$ if visits u, v are performed during the same trip, $l_{uv} = 1$ otherwise. We define $s(u, v) = \varphi d_{i_v i_u} + \psi l_{vu} + \chi \|qt_u - qt_v\|_1$, where φ, ψ, χ are independent scalars, and where the terms $d_{i_v i_u}$, resp. $\|qt_u - qt_v\|_1$ have been normalized by resp. the largest distance between a pair of customers, and a vector representing the largest demand for each product. The SR operator iteratively removes $\phi \in [1: \rho|V|]$ visits from the schedule, each time selecting the visit v^* which is most similar to the visit u removed in the preceding iteration, i.e. $v^* = \argmin_v s(u, v)$. The first visit to be removed is selected at random.

- Longest Route Removal(LRR): remove the longest route from the schedule.
- Visits in Long Routes Removal(VLRR): randomly remove $\phi \in [1, 0.5V_{num}]$ visits from the $\eta = 0.5|K|$ longest routes, where V_{num} is the total number of visits in these routes.

Next to the above removal operators, we use one additional operator which does not remove visits, but reallocates trips in their entirety:

- Trip Reallocation (TR): Randomly select and remove $\zeta \in [1: 0.45|T|]$ trips from the schedule. Sort the removed trips in descending order based on their duration, and iteratively re-insert them back into the schedule, each time assigning a trip to the vehicle with the shortest itinerary.

Note that, unlike the other removal operators, the TR operator is not followed by one of the insertion operators described below.

To repair the (partially) destroyed schedules, four repair operators are used to reschedule visits that are removed by the destroy operators:

- Greedy Insertion (GI): For each of the unscheduled visits in the removal pool, this operator computes the lowest cost to re-insert the visit somewhere in the schedule using the II, SI and STI insertion strategies. Next, the operator inserts the visit with the lowest insertion cost into the schedule, and updates the insertion costs of the remaining unscheduled visits accordingly. This procedure is repeated until all visits have been rescheduled.
- Greedy with Split Preference (GSP): Same as GI, but instead of considering all insertion strategies, this operator attempts to insert each visit using SI. Only in the event that a visit cannot be inserted through SI, e.g. when the existing trips do not have enough residual capacity, the operator resorts to the II and STI strategies.
- Regret Insertion (Ropke and Pisinger, 2006) (RI-2, RI-3): For each unscheduled visit v , we compute the cheapest option f_v^1 to reschedule visit v through the 3 different insertion strategies, as well as the i th cheapest alternative f_v^i for when options $f_v^1, \dots, f_v^{i-1}$ are unavailable. For each unscheduled visit v , a so-called regret cost Δ_v^i is computed, where Δ_v^i is the difference in cost between f_v^1 and f_v^i . At each iteration, the operator greedily selects the visit v with lowest cost Δ_v^i , reschedules it using f_v^1 , and updates f_v^j for $j = 1, \dots, i$ for the remaining unscheduled visits accordingly. We include two different Regret Insertion operators, RI-2 and RI-3, for $i = 2$ and $i = 3$ respectively.

Since the insertion heuristics are quite myopic, we followed the suggestion of Ropke and Pisinger (2006) to add noise to the insertion heuristics. This prevents these heuristics from always making the move that seems best locally. The same settings as in Ropke and Pisinger (2006) are used.

6. Experiments

In this section, we perform an experimental evaluation of the proposed ALNS heuristic based on a large testbed of FRP instances derived from real-world data. All experiments are conducted on a system having an Intel Core i5-3320M 2.67 GHz Processor, 8 GB RAM, and running Windows 10 Professional. The algorithms are implemented in Java 8. The MILP models are solved through IBM ILOG CPLEX 12.8 using default settings and a time limit of 7200 seconds. An overview of the parameters used in our ALNS implementation is provided in Table 5. These parameters are determined empirically. The scalars α , β in the objective function (2) are fixed to 100 and 1 respectively.

Table 5

The parameters of ALNS.

Notation	Description	Value
$nonImp_{max}$	Maximum number of non-improving iterations	1000
$timeLimit$	Time limit (in seconds)	3600
T	Initial temperature SA	200
κ	Cooling rate SA	0.9995
Nsg	Number of iterations per segment	100
ξ	Reaction factor	0.1
ρ	Destroy rate for removal operators	0.45
φ	First Shaw parameter	0.2
ψ	Second Shaw parameter	0.25
χ	Third Shaw parameter	0.1

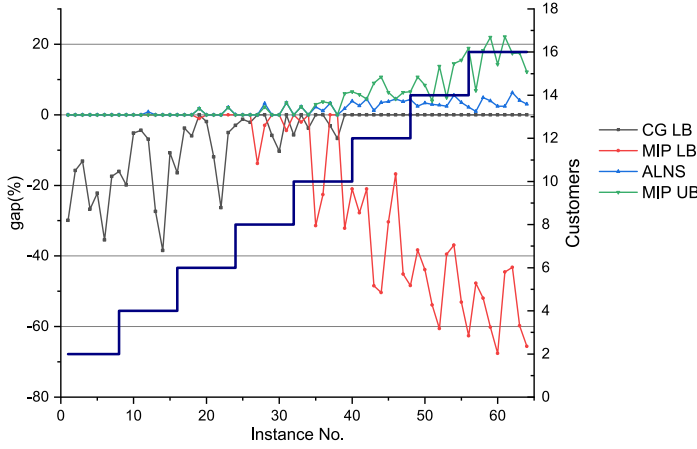
6.1. Problem instances

To generate realistic benchmark instances, we used data from NingBo Chaoyang Oil Transportation Co., Ltd., a Chinese petroleum transportation company. The customer demands for each product are sampled from their customer delivery data. Similarly, the vehicle sizes (number of compartments and compartment dimensions) are based on vehicles used by the company. Unfortunately, the company was unable to provide us with the actual locations of the fuel stations (customers), or the travel distances between each pair of locations. Instead we were provided with the travel distance r_i between the depot and each customer location $i \in V$. To mimic actual travel data, we assigned each customer i to a location relative to the depot by a polar coordinate (r_i, ϕ_i) , where ϕ_i is a random angle uniformly selected from $[0, 360]$. The distance d_{ij} between locations i and j is given by the Euclidean distance, rounded down to the nearest integer. By computing the distance matrix in this manner, we preserve the travel distances r_i , while guaranteeing that the distance matrix satisfies the triangle inequality.

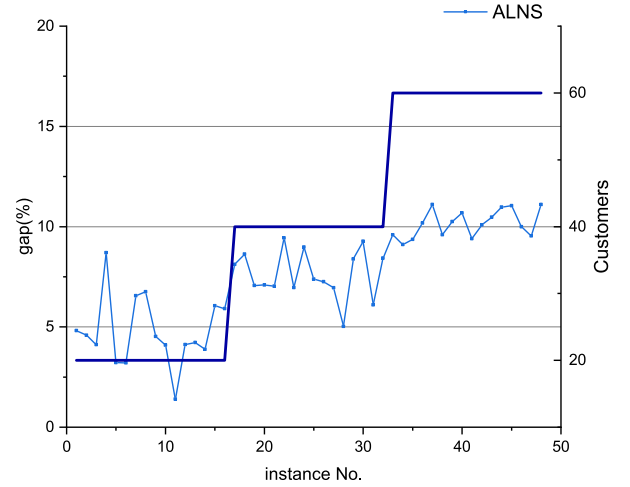
To evaluate the performance of the ALNS heuristic, we first generated 112 problem instances partitioned into 64 *small* and 48 *medium* instances. Each instance specifies a set of customers, their locations, a demand per product per customer, the number of available vehicles, the number of compartments in a vehicle, and the volume of a compartment. The *small* instances have up to 16 customers, up to 5 homogeneous vehicles each having 3 compartments with a fixed volume of 8k units, and up to 3 different petrol products. The *medium* instances on the other hand have up to 60 customers which are served by up to 20 homogeneous vehicles having the same number of compartments as in the small instances, but with capacities of up to 16k units. The following naming convention is used to uniquely identify these problem instances: $\alpha\text{-}v\beta\text{-}\gamma\text{-}c\delta - \epsilon$, where α denotes the number of customers, β the number of vehicles, γ the number of products, δ the number of compartments per vehicle and ϵ the capacity of a single compartment. Sections 6.2–6.5 present the computational results for these homogeneous problem instances. Later, in Section 6.6, we conduct a limited number of experiments on problem instances involving a heterogeneous fleet of vehicles. As described in Section 6.6, these heterogeneous problem instances are obtained through a simple transformation of the homogeneous problem instances.

6.2. Performance of ALNS

In this section, we analyze the performance of the ALNS heuristic by comparing it against the bounds produced by the MILP model from Section 3 and the CG model from Section 4. We solve the set of *small* instances with the MILP model, the CG model and the ALNS heuristic. The *medium* instances are solved through CG and ALNS since these instances are too large to solve through the MILP formulation.



(a) Small instances



(b) Medium instances

Fig. 2. Gap comparison.

Detailed results for each instance are given in Tables A.7 and A.8 in the Appendix. Column LB provides the *strongest* lower bound derived from the CG and, when available, the MILP model. For ALNS, we report the best f_b , average f_a and worst f_w solution observed in 10 independent invocations of the ALNS algorithm, as well as the average computation time $t(s)$. Column *gap* reports the percentage gap between f_b and LB, calculated as $\frac{f_b - LB}{f_b}$. For MILP, we report the best solution f obtained within a time limit of 2 hours, the percentage gap between f and LB, and the computation time $t(s)$ in seconds. Finally, for CG, we report the best objective value f and the computation time $t(s)$ in seconds. All instances are ordered in ascending order of the number of customers, number of vehicles, number of products, number of compartments and compartment size.

As can be observed from Table A.7, the MILP model only solves instances up to 10 customers within a time limit of 2h. Out of the 64 small instances, MILP solves 18 instances to proven optimality. The same optimal solutions are found by our ALNS heuristic. For the instances with 12 or more customers, MILP never terminates before the two-hour time limit. The ALNS heuristic however is able to find high quality solutions (average gap 1.7%) for all small instances in less than 10 seconds per instance. When comparing the best ALNS solutions $f(b)$ with the MILP solutions f we observe that ALNS consistently outperforms MILP as the ALNS solutions are either identical or better than the MILP solutions for all small instances.

To assess the scalability of the ALNS heuristic, we repeat these computational experiments on the set of medium instances (Table A.8). For these instances, we can only compare against the bounds obtained from our CG model. Even for the largest instances in this set, having 60 customers and 20 vehicles, computation times of the ALNS heuristic stay well below 25 minutes, while obtaining an average gap ($\frac{f_b - LB}{f_b}$) of 7.5%. When expressing the gap between LB and average objective value f_a (as opposed to f_b), we witness only a minor gap increase of 1.3%. This shows that the ALNS results are robust across multiple runs.

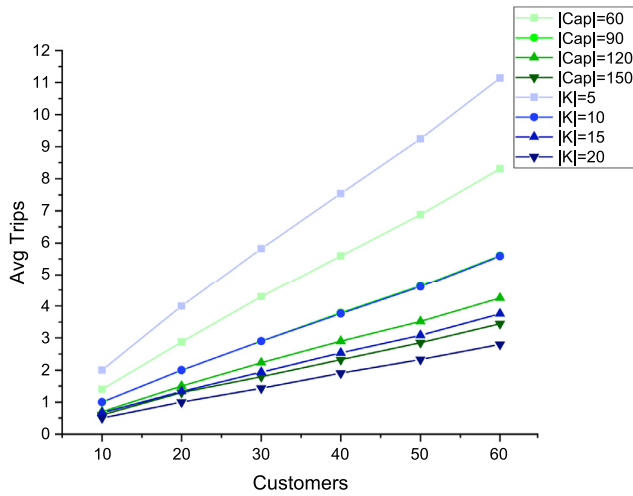
Fig. 2a depicts a summary of the bounds computed for each of the instances in the *small* data set. The 0% gap line corresponds to the LB column in Table A.7, i.e. the strongest lower bound. The blue line (ALNS) and green line (MILP UB) correspond to the

gap(%) columns in the same table. Finally, the black line (CG LB) and the red line (MILP LB) represent the gaps between these two lower bounds and the strongest lower bound (0% line). For instance, the black line is calculated as $-\frac{LB - CG}{LB}$. As can be observed from Fig. 2, for the first 18 instances the MILP model produces the strongest lower bounds. Once the instances become too large (12 customers or more), the MILP model is no longer able to derive strong bounds; for instances 40–64 the best lower bound is provided by the CG model. Fig. 2a depicts the gaps computed between ALNS and CG for the medium instances. Even for the largest instances in this set, the gap stays well below 12%.

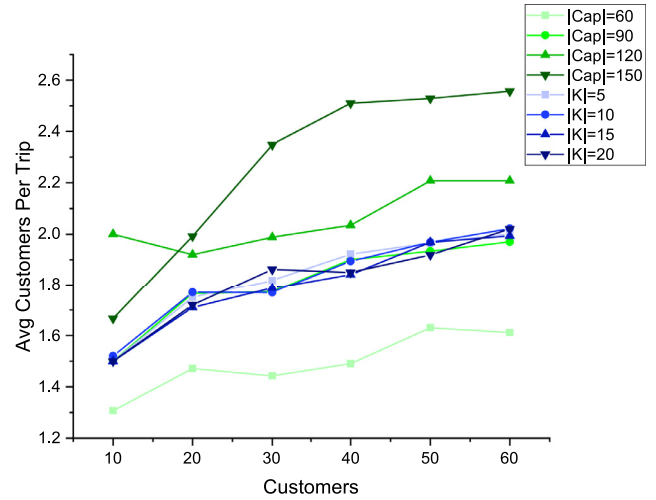
Fig. 3 analyses various properties of the solutions obtained through the ALNS heuristic. $[Cap]$ is the total vehicle capacity (sum of compartment capacities). Fig. 3a shows the average number of trips performed by a vehicle. This number ranges from 0.5 to 10.5, and tends to grow linear in the number of customers. The slope of these curves is proportional to the number of available vehicles, and their capacities. Fig. 3b depicts the average number of customers visited during a single trip. Clearly this number is limited by the vehicle capacity. In the FRP the average number of customers per trip is quite low due to the fact that customer demands are high compared to the vehicle capacities. Finally Fig. 3c measures the vehicle utilization, computed as the average fraction of the vehicle capacity occupied during a trip. Clearly, the ALNS heuristic is able to compute very dense solutions, with average utilization rates of up to 97%. The average utilization rate increases inversely proportional to the total vehicle capacity, and is relatively independent of the number of available vehicles.

6.3. Computation times

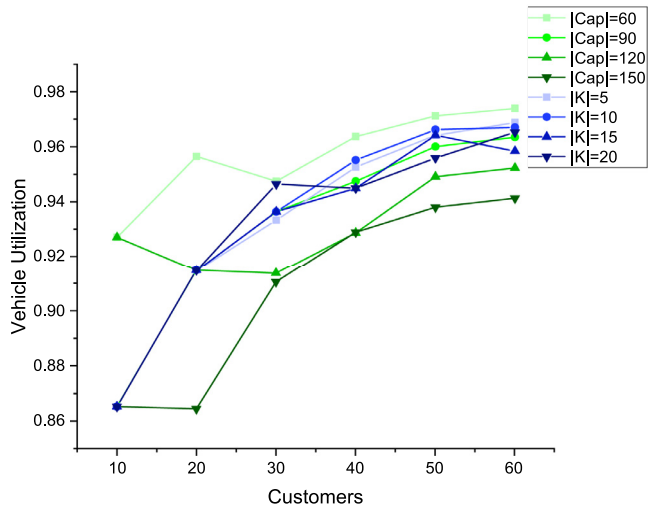
Fig. 4 shows the influence of the various instance features on the average ALNS computation times. Naturally, computation times go up when the number of customers increases. For a fixed number of customers, we observe a *decrease* in computation time when the number of vehicle compartments or their capacity increases (Fig. 4b and c). A strong *increase* in computation time is observed when the number of different products increases (Fig. 4d). Finally, in contrast to the aforementioned instance features, we do not witness a strong correlation between the number of vehicles and the total computation time of the ALNS heuristic (Fig. 4a).



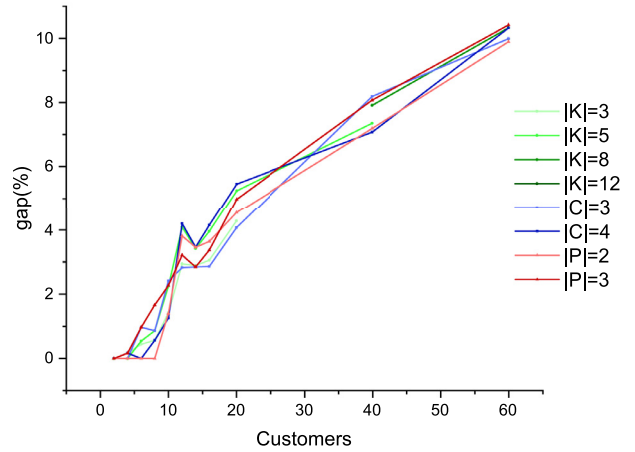
(a) Average number of trips per vehicle.



(b) Average number of customers visited per trip.



(c) Average vehicle capacity utilization.



(d) Average gap.

Fig. 3. Solution statistics.

6.4. Impact of compartment splitting

Individual compartments of fuel delivery vehicles are typically equipped with debit meters to deliver precise quantities of fuel to customers. Moreover, it becomes possible to split the fuel stored into a *single* compartment over multiple customers. From an operational as well as a computational perspective, allowing compartment splitting increases the complexity of the problem due to a drastic increase in search space. A reasonable question to ask is how beneficial it is to allow compartment splitting? One might conjecture that there is a diminishing effect to compartment splitting when the number of customers increases due to an increase in the number of potential routes and delivery patterns.

A comparison of the results obtained through ALNS *with* and *without* compartment-splitting over multiple customers on a subset of 32 instances containing 20–40 customers is presented in Fig. 5. As is shown in the figure, the objective value of all 32 instances improves when compartment-splitting is allowed. The average improvement in objective value equals 12.5%. Moreover, we observed a marginal increase in the average objective improvement

obtained from splitting compartments when the number of customers increases.

6.5. ALNS Operator performance

Fig. 6 investigates the performance of the ALNS operators described in Section 5.3. This analysis is performed using average results obtained from 16 instances, each having 40 customers, and which have been solved 10 times per instance. The x-axis of each graph shows the progress of the heuristic: the heuristic starts at 0% 'completion', and finishes at 100%. This completion metric allows us to compare different ALNS runs mutually, independent of the number of iterations performed by the time the heuristic terminates. Fig. 6a depicts the cumulative contribution of each operator, expressed in the number of times an operator discovered a new incumbent solution and normalized by the total number of times a new incumbent solution was found (percentage). Solid lines represent destroy operators, whereas dashed lines are the repair operators. Naturally, the largest number of improving solutions are discovered early in the search. Towards the end of the search, the

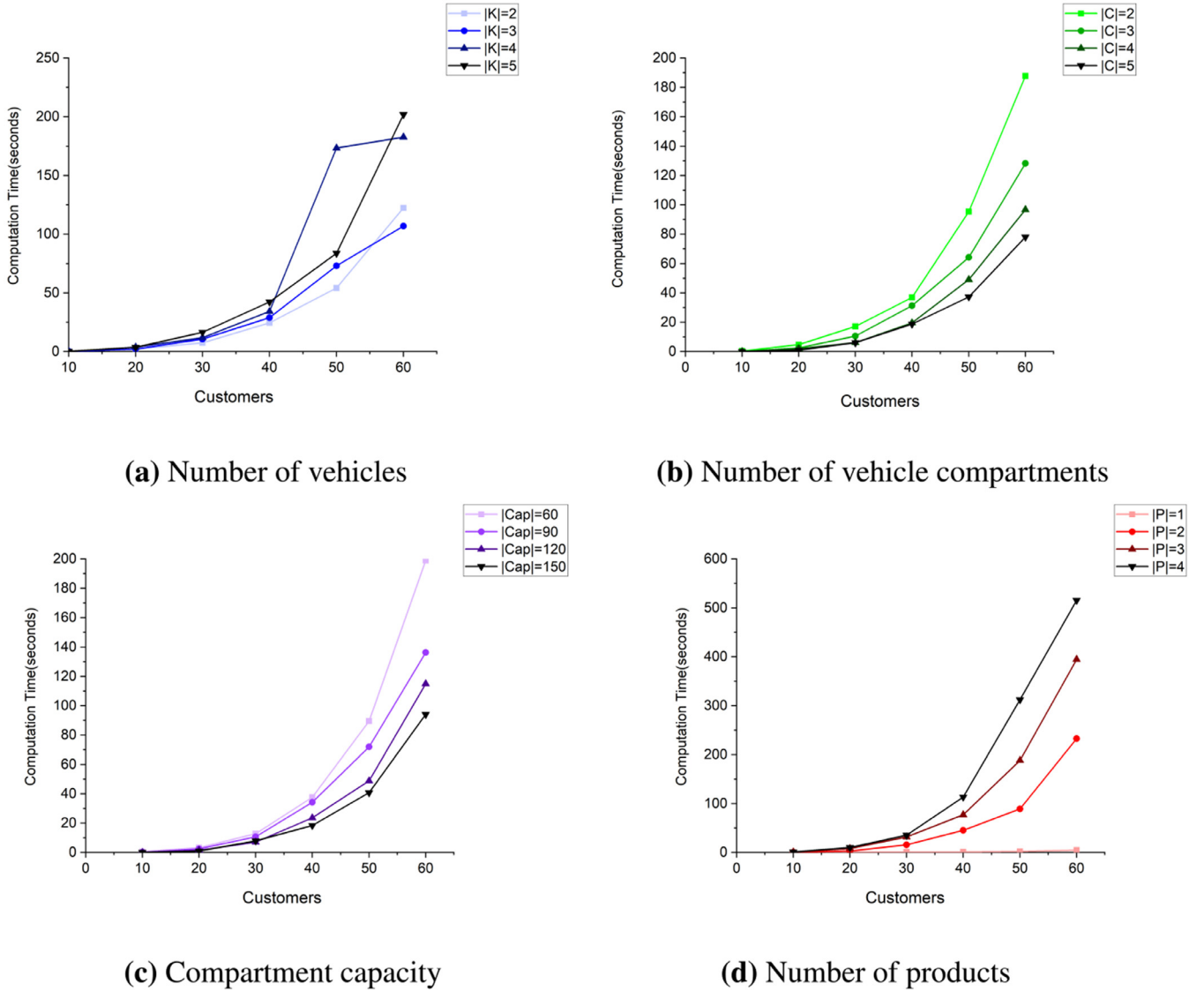


Fig. 4. Impact of instance features on average ALNS computation time.

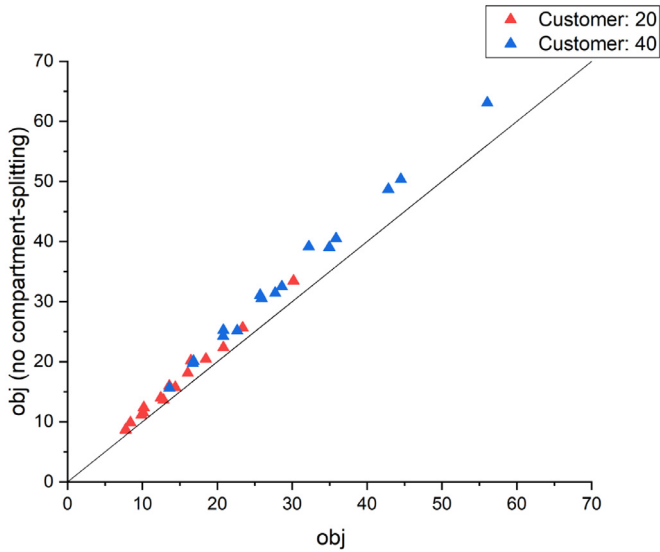


Fig. 5. Impact of compartment splitting. Each triangle represents an instance from the medium data set; the triangle colors correspond to the number of customers in the instance. A slight increase in the benefit obtained through compartment splitting can be observed when the number of customers increases.

graph gradually flattens out. The SR destroy operator (Shaw Removal, green) and the RI-3 repair operator (Remove-Insert, brown) are responsible for the discovery of the largest number of new best solutions.

Recall from Section 5.2 that during each ALNS iteration, the destroy and repair operators are selected proportional to their normalized performance $w_\gamma / \sum_{i \in \Gamma} w_i$, and that the performance of an operator $\gamma \in \Gamma$ is measured in terms of collected rewards (Table 4). The graphs in Fig. 6b and 6c plot for each operator $\gamma \in \Gamma$ the average ratio $w_\gamma / \sum_{i \in \Gamma} w_i$ over the course of the heuristic. Interesting to observe is that although percentage wise the SR removal operator discovers the largest number of incumbent solutions (Fig. 6a), the performance of this operator degrades over time (Fig. 6b) and is surpassed by the LRR (black) and VLRR operators (pink). The RI-2, RI-3 repair operators consistently outperform the GI and GSP operators (Fig. 6c). Finally we observe from Fig. 6a and b that the performance of the Worst-Removal (WR, green) operator is relatively poor. Also in the work by Ropke and Pisinger (2006), this operator has the lowest performance. To investigate whether it would be better to disable the WR operator altogether, we reran our experiments. We observed that disabling the WR operator benefits some instances, but negatively affects other instances. We suspect that this operator is useful in some cases to escape

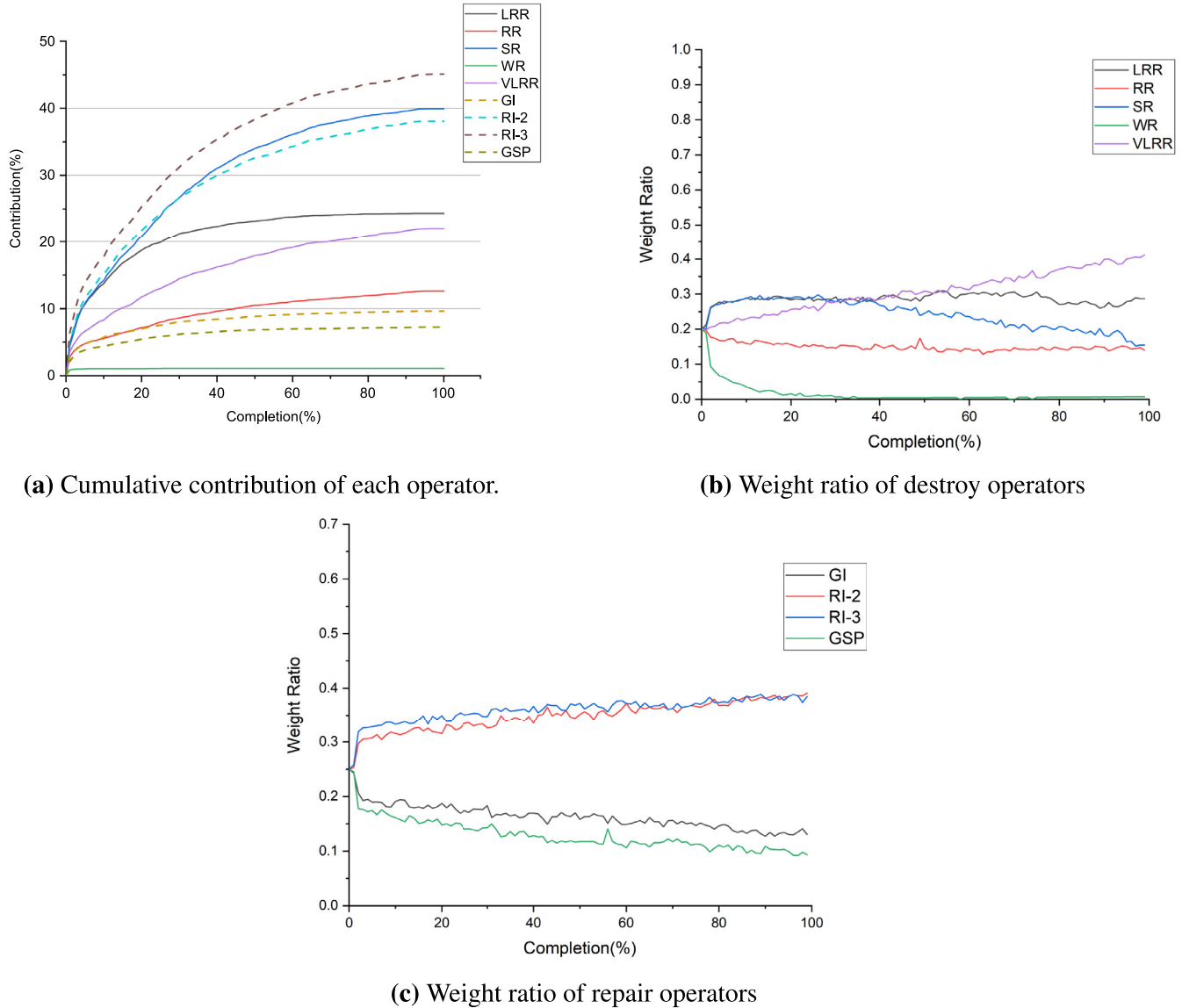


Fig. 6. ALNS operator performance, averaged over 10 invocations of the ALNS heuristic over 16 instances with 40 customers each.

local minima. Therefore, we decided to leave the operator in, and let the operator selection mechanism (Section 5.2) deal with it.

6.6. Heterogeneous vehicles

Thus far, all experiments are conducted with a homogeneous fleet of vehicles. In this subsection we investigate the performance of the ALNS heuristic on problem instances involving a heterogeneous set of vehicles. To perform this experiment in a controlled manner, we use the homogeneous instances introduced in Section 6.1 to generate sets of heterogeneous problem instances.

In what remains we refer to the set of homogeneous instances as Type A instances. We generate two new sets of heterogeneous problem instances, which we will refer to as Type B and Type C instances respectively. These new instances only differ in one dimension from the Type A instances: instances of Type B are instances in which the vehicle capacity of all the vehicles is the same, but some vehicles have more (but smaller) compartments,

whereas for the Type C instances all vehicles have the same number of equal-sized compartments, but some vehicles are larger than others. Note that we do not consider problem instances where some vehicles have unequal sized compartments, as no such vehicles exist in our real-world data. Moreover, the presence of vehicles with unequal sized compartments would further complicate the problem at hand, since we would have to explicitly determine to which compartment a product is assigned, as opposed to assigning products to vehicles without distinguishing individual compartments.

Given an instance of type A with k vehicles, C compartments per vehicle, and Q the capacity of a single compartment, and $\bar{Q} = C \times Q$ the total vehicle capacity, we create instances of types B and C as follows:

Type B: $k_1 = \lfloor 0.75 \times k \rfloor$ vehicles with $C_1 = C - 1$ compartments with a compartment size of $Q_1 = \bar{Q} / (C - 1)$, and $k_2 = \lceil 0.25 \times k \rceil$ vehicles with $C_2 = C$ compartments with a compartment size of $Q_2 = \bar{Q} / C$

Table 6
Results for the heterogeneous problem instances (averages).

	Small instances			Medium instances		
	A	B	C	A	B	C
f_b	8769.84	8661.11	8656.70	24925.40	24645.71	24673.69
f_a	8827.50	8739.25	8766.04	25264.17	25007.96	25034.56
f_w	8935.86	8842.52	8878.59	25675.85	25425.00	25456.71
gap	1.69	6.01	5.89	7.50	8.24	9.43
t(s)	0.61	1.02	1.05	99.47	201.37	213.93
AVG trips/vehic	3.00	2.96	2.96	5.08	5.07	5.12
AVG cust/trip	1.41	1.41	1.37	2.21	2.13	2.12
AVG vehic cap util	0.89	0.89	0.88	0.93	0.93	0.93

Type C: $k1 = \lfloor 0.75 \times k \rfloor$ vehicles with $C1 = C$ compartments with a compartment size of $Q1 = 0.75\bar{Q}/C$, and $k2 = \lceil 0.25 \times k \rceil$ vehicles with $C2 = C$ compartments with a compartment size of $Q2 = \frac{\bar{Q} \times k - (k1 \times C \times Q1)}{C \times k2}$

Note that, for a given instance of Type A, the corresponding instances of Types B and C have the exact same number of vehicles, and, summed over all vehicles, the total available vehicle capacity is also the same. As per example, from a type A instance s20-v5-p3-c3-8000 with $k = 5$, $C = 3$, $Q = 8000$, the following 2 new instances are created:

Type B: $k1 = 3$ vehicles with $C1 = 2$ compartments with a compartment size of $Q1 = 12000$, and $k2 = 2$ vehicles with $C2 = 3$ compartments with a compartment size of $Q2 = 8000$. Notice that both the vehicles with 2 compartments and with 3 compartments have a total capacity of 24000. The 5 vehicles together have $5 \times 24000 = 120000$ capacity.

Type C: $k1 = 3$ vehicles with $C1 = 3$ compartments with a compartment size of $Q1 = 6000$, and $k2 = 2$ vehicles with $C2 = 3$ compartments with a compartment size of $Q2 = 11000$. So we have 3 smaller vehicles with a total vehicle capacity of 18000, and 2 larger vehicles with a total vehicle capacity of 33000. These 5 vehicles together have 120000 capacity.

Using the Type A instances listed in Tables A.7 and A.8, we generate both small and medium sized instances of Types B and C. Table 6 reports the results averaged over all heterogeneous instances. A description of the reported values can be found in Section 6.2.

When comparing the results of Type B, C against Type A, we notice that both the best, average and worst ALNS objective values improve while the optimality gaps stay roughly the same. Despite the fact that the heterogeneous instances are harder to solve, as witnessed from the increased computation times, we conclude that the ALNS heuristic produces solutions of consistent quality across different instance types. Increasing the number of compartments per vehicle, as is the case for instances of Type B, results in a slightly higher vehicle utilization, and hence a marginal reduction in the number of trips that have to be performed per vehicle. The improvements in both the best, average and worst ALNS solutions for Type C, although minor, show that further cost reductions can be achieved when an adequate mixture of differently-sized vehicles is deployed.

7. Conclusions

In this paper, we addressed a Fuel Replenishment Problem (FRP). Specifically, we considered a split delivery vehicle routing problem with multiple compartments and multiple trips. The objective is to determine the routing of the vehicles, the delivery pattern of each trip and the allocation of products to vehicle compartments, while minimizing the makespan of the resulting routes. We formally defined the problem and proposed a MILP model for it. In order to solve this problem effectively, we proposed an Adaptive Large Neighborhood Search (ALNS) algorithm. Additionally, we also proposed a column generation approach to compute tight lower bounds.

We generated small and medium-sized instance sets to evaluate the performance of ALNS. We showed that the ALNS is able to find (near) optimal solutions much faster than the exact MILP model using CPLEX. Moreover, the ALNS heuristic is able to handle instances that are significantly larger: we obtain very good solutions within 1000 seconds for instances with 60 customers, 20 vehicles, 3 products and 4 compartments. Finally, sensitivity analysis is performed to assess the influence of important problem instance features (number of vehicles, products, compartments and their capacity) on the solution quality and computation times. Our numerical results show that significant savings are obtained by allowing for multiple trips and splitting customer demand.

Future research could involve extending our models and solution approach to consider stochastic demands. In this context, Inventory-Routing models are highly relevant to consider.

CRediT authorship contribution statement

L. Wang: Conceptualization, Methodology, Software, Validation, Writing - original draft. **J. Kinable:** Conceptualization, Methodology, Software, Validation, Writing - original draft, Supervision. **T. van Woensel:** Conceptualization, Writing - review & editing, Supervision.

Acknowledgements

This research was supported in part by the Netherlands Organisation for Scientific Research (NWO) grant 629.002.211.

Appendix A. Detailed computational results

Table A1
Small instances.

instance	LB	ALNS					MILP			CG	
		f_b	f_a	f_w	gap	t(s)	f	gap	t(s)	f	t(s)
s2-v3-p2-c3-8000	4900.0	4900	4900	4900	0.0	0.2	4900	0.0	4.6	3433.3	0.1
s2-v3-p2-c4-8000	3383.0	3383	3383	3383	0.0	0.2	3383	0.0	3.3	2849.7	0.1
s2-v3-p3-c3-8000	5335.0	5335	5335	5335	0.0	0.2	5335	0.0	2.4	4635.0	0.1
s2-v3-p3-c4-8000	5109.0	5109	5109	5109	0.0	0.1	5109	0.0	3.9	3742.3	0.2
s2-v5-p2-c3-8000	2700.0	2700	2700	2700	0.0	0.1	2700	0.0	2.4	2100.0	0.1
s2-v5-p2-c4-8000	2700.0	2700	2700	2700	0.0	0.0	2700	0.0	2.5	1743.0	0.1
s2-v5-p3-c3-8000	3435.0	3435	3435	3435	0.0	0.1	3435	0.0	2.9	2835.0	0.1
s2-v5-p3-c4-8000	2726.0	2726	2726	2726	0.0	0.1	2726	0.0	2.7	2289.0	0.1
s4-v3-p2-c3-8000	7372.0	7372	7372	7372	0.0	0.1	7372	0.0	6.7	5905.3	0.2
s4-v3-p2-c4-8000	5249.0	5249	5249	5249	0.0	0.1	5249	0.0	2.6	4978.3	0.3
s4-v3-p3-c3-8000	9875.0	9875	9875	9875	0.0	0.2	9875	0.0	320.7	9441.7	0.4
s4-v3-p3-c4-8000	7339.0	7399	7459	7599	0.8	0.1	7339	0.0	5.5	6832.3	0.3
s4-v5-p2-c3-8000	4972.0	4972	4972	4972	0.0	0.1	4972	0.0	6.7	3612.0	0.2
s4-v5-p2-c4-8000	4949.0	4949	4949	4949	0.0	0.1	4949	0.0	4.3	3045.0	0.3
s4-v5-p3-c3-8000	6476.0	6476	6555.9	7275	0.0	0.1	6476	0.0	242.5	5775.0	0.4
s4-v5-p3-c4-8000	5003.0	5003	5023.2	5205	0.0	0.1	5003	0.0	6.2	4179.0	0.6
s6-v3-p2-c3-8000	7410.0	7410	7410	7410	0.0	0.1	7410	0.0	19.1	7129.7	0.6
s6-v3-p2-c4-8000	6375.0	6375	6375	6375	0.0	0.1	6375	0.0	9.0	5995.3	0.5
s6-v3-p3-c3-8000	11126.9	11327	11378.1	11,534	1.8	0.2	11327	1.8	7204.8	11126.9	1.0
s6-v3-p3-c4-8000	8649.0	8649	8649.9	8650	0.0	0.1	8649	0.0	426.9	8480.3	2.3
s6-v5-p2-c3-8000	5010.0	5010	5010	5010	0.0	0.1	5010	0.0	72.0	4410.0	0.8
s6-v5-p2-c4-8000	4979.0	4979	4979	4979	0.0	0.1	4979	0.0	123.9	3667.1	0.7
s6-v5-p3-c3-8000	7083.6	7236	7275.6	7335	2.1	0.2	7236	2.1	7210.1	6727.0	1.1
s6-v5-p3-c4-8000	5347.0	5349	5409.2	5550	0.0	0.1	5347	0.0	397.3	5187.0	2.9
s8-v3-p2-c3-8000	8547.0	8547	8547	8547	0.0	0.2	8547	0.0	483.3	8438.8	1.0
s8-v3-p2-c4-8000	7515.0	7515	7537	7619	0.0	0.1	7515	0.0	184.1	7356.1	0.9
s8-v3-p3-c3-8000	12978.0	12978	13088.3	13,181	0.0	0.4	12978	0.0	7205.1	12978.0	4.4
s8-v3-p3-c4-8000	10065.7	10400	10419.7	10,500	3.2	0.2	10297	2.2	7200.6	10065.7	5.8
s8-v5-p2-c3-8000	5450.0	5450	5502.1	5647	0.0	0.1	5450	0.0	3293.2	5130.3	1.4
s8-v5-p2-c4-8000	5019.0	5019	5020.8	5022	0.0	0.1	5019	0.0	208.6	4504.2	1.8
s8-v5-p3-c3-8000	7800.0	8078	8088.6	8178	3.4	0.4	8078	3.4	7201.2	7800.0	6.3
s8-v5-p3-c4-8000	6602.0	6602	6671.3	6705	0.0	0.2	6602	0.0	4443.3	6226.5	4.9
s10-v3-p2-c3-8000	9357.6	9577	9751.8	9883	2.3	0.3	9577	2.3	7203.6	9357.6	2.1
s10-v3-p2-c4-8000	8237.0	8237	8257.6	8342	0.0	0.2	8237	0.0	765.6	7923.5	3.1
s10-v3-p3-c3-8000	14093.0	14420	14,440	14,520	2.3	0.8	14,523	3.0	7217.9	14093.0	5.8
s10-v3-p3-c4-8000	11306.4	11432	11554.5	11,636	1.1	0.4	11,736	3.7	7209.3	11306.4	19.5
s10-v5-p2-c3-8000	5876.9	6077	6090.6	6189	3.3	0.3	6077	3.3	7200.2	5693.3	2.5
s10-v5-p2-c4-8000	5238.0	5238	5238.1	5239	0.0	0.2	5238	0.0	565.7	4886.2	2.4
s10-v5-p3-c3-8000	8663.0	8820	8850	8920	1.8	0.7	9220	6.0	7220.4	8663.0	12.6
s10-v5-p3-c4-8000	6853.4	7132	7153.1	7235	3.9	0.4	7335	6.6	7215.4	6853.4	10.5
s12-v3-p2-c3-8000	11740.9	12049	12059.4	12,150	2.6	0.4	12,461	5.8	7223.1	11740.9	12.3
s12-v3-p2-c4-8000	9445.5	9888	9917.9	9988	4.5	0.3	9888	4.5	7213.9	9445.5	7.8
s12-v3-p3-c3-8000	17899.2	18126	18178.1	18,230	1.3	2.2	19,672	9.0	7206.8	17899.2	35.2
s12-v3-p3-c4-8000	13421.9	13904	14025.9	14,107	3.5	0.8	15,034	10.7	7204.3	13421.9	23.6
s12-v5-p2-c3-8000	7266.0	7549	7590.8	7750	3.7	0.5	7757	6.3	7208.6	7266.0	8.7
s12-v5-p2-c4-8000	5911.7	6188	6241	6391	4.5	0.4	6189	4.5	7205.9	5911.7	8.9
s12-v5-p3-c3-8000	10712.5	11128	11188.8	11,231	3.7	2.8	11,437	6.3	7211.8	10712.5	27.0
s12-v5-p3-c4-8000	8229.4	8607	8647.4	8711	4.4	1.0	8805	6.5	7213.8	8229.4	35.6
s14-v3-p2-c3-8000	12859.2	13183	13223.9	13,284	2.5	0.5	14,399	10.7	7214.6	12859.2	7.5
s14-v3-p2-c4-8000	10648.9	11018	11029.2	11,120	3.4	0.6	11,627	8.4	7209.9	10648.9	16.6
s14-v3-p3-c3-8000	19387.0	19981	20082.7	20,388	3.0	4.4	20,184	3.9	7208.7	19387.0	37.1
s14-v3-p3-c4-8000	14821.0	15243	15,263	15,343	2.8	2.0	17,188	13.8	7213.1	14821.0	59.6
s14-v5-p2-c3-8000	7983.8	8184	8194.1	8285	2.4	0.6	8392	4.9	7205.8	7983.8	31.2
s14-v5-p2-c4-8000	6439.3	6819	6898.7	7118	5.6	0.6	7528	14.5	7210.6	6439.3	6.3
s14-v5-p3-c3-8000	11852.4	12281	12301.9	12,386	3.5	3.4	14,030	15.5	7212.4	11852.4	38.1
s14-v5-p3-c4-8000	9142.1	9343	9454.2	9548	2.2	1.4	11,264	18.8	7210.9	9142.1	61.1
s16-v3-p2-c3-8000	14380.4	14522	14562.5	14,623	1.0	0.9	15,443	6.9	7210.6	14380.4	45.0
s16-v3-p2-c4-8000	11366.2	11947	11978.1	12,050	4.9	0.9	13,899	18.2	7211.1	11366.2	19.2
s16-v3-p3-c3-8000	20866.8	21731	21916.3	22,141	4.0	6.7	26,740	22.0	7209.8	20866.8	121.3
s16-v3-p3-c4-8000	15880.6	16272	16426.4	16,685	2.4	3.0	18,534	14.3	7224.7	15880.6	50.7
s16-v5-p2-c3-8000	8706.1	8922	8962.6	9122	2.4	1.2	11,177	22.1	7216.1	8706.1	25.9
s16-v5-p2-c4-8000	6977.6	7447	7508.9	7650	6.3	0.9	8449	17.4	7225.8	6977.6	20.8
s16-v5-p3-c3-8000	12789.3	13331	13476.9	13,640	4.1	6.2	15,510	17.5	7216.3	12789.3	55.2
s16-v5-p3-c4-8000	9766.5	10072	10169.9	10,388	3.0	2.9	11,119	12.2	7209.3	9766.5	63.1

Table A2
large instances.

Instance	LB	ALNS				
		f_b	f_a	f_w	gap(%)	t(s)
s20-v5-p2-c3-8000	12188.5	12,806	12909.6	13,115	4.8	2.8
s20-v5-p2-c3-16000	7409.5	7765	8015	8184	4.6	1.2
s20-v5-p2-c4-8000	9462.0	9868	10010.8	10,287	4.1	2.0
s20-v5-p2-c4-16000	7087.8	7763	7795.3	7870	8.7	1.7
s20-v5-p3-c3-8000	17881.4	18,476	18737.1	19,000	3.2	9.7
s20-v5-p3-c3-16000	9854.0	10,181	10,501	10,702	3.2	3.4
s20-v5-p3-c4-8000	13438.6	14,381	14557.1	14,797	6.6	5.1
s20-v5-p3-c4-16000	7826.4	8393	8601.7	8914	6.8	2.9
s20-v3-p2-c3-8000	19863.7	20,806	20937.9	21,112	4.5	2.0
s20-v3-p2-c3-16000	11951.3	12,461	12625.4	12,770	4.1	1.4
s20-v3-p2-c4-8000	15843.3	16,068	16159.2	16,377	1.4	1.3
s20-v3-p2-c4-16000	9671.3	10,087	10212.4	10,297	4.1	0.8
s20-v3-p3-c3-8000	28904.7	30,178	30465.6	30,896	4.2	8.9
s20-v3-p3-c3-16000	15840.2	16,480	16848.3	17,301	3.9	3.5
s20-v3-p3-c4-8000	21964.4	23,380	23646.4	23,892	6.1	5.1
s20-v3-p3-c4-16000	12789.5	13,593	13788.3	14,205	5.9	2.4
s40-v8-p2-c3-8000	26291.0	28,614	28959.1	29,348	8.1	52.7
s40-v8-p2-c3-16000	15283.1	16,726	17125.5	17,365	8.6	26.2
s40-v8-p2-c4-8000	21064.5	22,666	22906.6	23,308	7.1	42.8
s40-v8-p2-c4-16000	12637.0	13,601	13799.5	14,086	7.1	14.2
s40-v8-p3-c3-8000	33332.9	35,855	36513.8	37,031	7.0	109.2
s40-v8-p3-c3-16000	18848.1	20,811	21144.2	21,373	9.4	57.7
s40-v8-p3-c4-8000	25807.2	27,737	28040.5	28,710	7.0	81.2
s40-v8-p3-c4-16000	15313.6	16,824	17167.5	17,646	9.0	31.4
s40-v5-p2-c3-8000	41228.1	44,515	44891.1	45,773	7.4	36.1
s40-v5-p2-c3-16000	24041.8	25,923	26430.8	26,979	7.3	19.6
s40-v5-p2-c4-8000	32531.6	34,962	35338.2	35,898	7.0	32.8
s40-v5-p2-c4-16000	19740.2	20,786	21175.9	21,413	5.0	15.0
s40-v5-p3-c3-8000	51363.0	56,067	56652.1	57,118	8.4	106.4
s40-v5-p3-c3-16000	29233.9	32,220	32573.9	32,969	9.3	53.9
s40-v5-p3-c4-8000	40220.4	42,832	43505.7	44,094	6.1	63.7
s40-v5-p3-c4-16000	23551.8	25,719	26084.4	26,450	8.4	32.4
s60-v8-p2-c3-8000	41780.3	46,210	46638.9	47,495	9.6	258.1
s60-v8-p2-c3-16000	23941.0	26,339	26728.4	27,268	9.1	101.9
s60-v8-p2-c4-8000	32678.6	36,056	36521.4	37,241	9.4	157.3
s60-v8-p2-c4-16000	19278.8	21,463	21752.3	22,078	10.2	76.2
s60-v8-p3-c3-8000	52705.7	59,290	59725.3	60,357	11.1	580.4
s60-v8-p3-c3-16000	29862.5	33,034	33609.6	34,542	9.6	236.7
s60-v8-p3-c4-8000	40697.6	45,345	45897.1	46,424	10.2	411.9
s60-v8-p3-c4-16000	23518.1	26,332	26713.7	27,155	10.7	197.2
s60-v12-p2-c3-8000	28885.1	31,882	32295.4	32,797	9.4	235.3
s60-v12-p2-c3-16000	16554.0	18,411	18611.4	18,885	10.1	111.6
s60-v12-p2-c4-8000	22593.3	25,235	25444.7	25,821	10.5	149.6
s60-v12-p2-c4-16000	13319.0	14,960	15168.7	15,487	11.0	77.0
s60-v12-p3-c3-8000	36437.0	40,961	41531.1	41,945	11.0	536.3
s60-v12-p3-c3-16000	20636.7	22,929	23362.9	24,107	10.0	281.8
s60-v12-p3-c4-8000	28136.6	31,103	31851.4	32,211	9.5	340.5
s60-v12-p3-c4-16000	16289.6	18,325	18707.8	19,348	11.1	193.7

References

- Abdelaziz, F.B., Roucairol, C., Bacha, C., 2002. Deliveries of liquid fuels to snpd gas stations using vehicles with multiple compartments. In: IEEE international conference on systems, man and cybernetics, 1. IEEE, pp. 478–483.
- Archetti, C., Bouchard, M., Desaulniers, G., 2011. Enhanced branch and price and cut for vehicle routing with split deliveries and time windows. *Transp. Sci.* 45 (3), 285–298.
- Archetti, C., Speranza, M.G., Hertz, A., 2006. A tabu search algorithm for the split delivery vehicle routing problem. *Transp. Sci.* 40 (1), 64–73.
- Avella, P., Boccia, M., Sforza, A., 2004. Solving a fuel delivery problem by heuristic and exact approaches. *Eur. J. Oper. Res.* 152 (1), 170–179.
- Benantar, A., Ouafi, R., Boukachour, J., 2016. A petrol station replenishment problem: new variant and formulation. *Logist. Res.* 9 (1), 6.
- Coelho, L.C., Laporte, G., 2015. Classification, models and exact algorithms for multi-compartment delivery problems. *Eur. J. Oper. Res.* 242 (3), 854–864.
- Cornillier, F., Boctor, F., Renaud, J., 2012. Heuristics for the multi-depot petrol station replenishment problem with time windows. *Eur. J. Oper. Res.* 220 (2), 361–369.
- Cornillier, F., Boctor, F.F., Laporte, G., Renaud, J., 2008. An exact algorithm for the petrol station replenishment problem. *J. Oper. Res. Soc.* 59 (5), 607–615.
- Cornillier, F., Boctor, F.F., Laporte, G., Renaud, J., 2008. A heuristic for the multi-petrol station replenishment problem. *Eur. J. Oper. Res.* 191 (2), 295–305.
- Cornillier, F., Laporte, G., Boctor, F.F., Renaud, J., 2009. The petrol station replenishment problem with time windows. *Comput. Oper. Res.* 36 (3), 919–935.
- Derigs, U., Gottlieb, J., Kalkoff, J., Piesche, M., Rothlauf, F., Vogel, U., 2011. Vehicle routing with compartments: applications, modelling and heuristics. *OR spectrum* 33 (4), 885–914.
- Desaulniers, G., 2010. Branch-and-price-and-cut for the split-delivery vehicle routing problem with time windows. *Oper. Res.* 58 (1), 179–192.
- Dror, M., Trudeau, P., 1989. Savings by split delivery routing. *Transp. Sci.* 23 (2), 141–145.
- Dror, M., Trudeau, P., 1990. Split delivery routing. *Naval Res. Logist. (NRL)* 37 (3), 383–402.
- El Fallahi, A., Prins, C., Calvo, R.W., 2008. A memetic algorithm and a tabu search for the multi-compartment vehicle routing problem. *Comput. Oper. Res.* 35 (5), 1725–1741.
- Feillet, D., Dejax, P., Gendreau, M., Gueguen, C., 2006. Vehicle routing with time windows and split deliveries. *Tech. Paper* 851.
- Henke, T., Speranza, M.G., Wäscher, G., 2015. The multi-compartment vehicle routing problem with flexible compartment sizes. *Eur. J. Oper. Res.* 246 (3), 730–743.
- Hübner, A., Ostermeier, M., 2018. A multi-compartment vehicle routing problem with loading and unloading costs. *Transp. Sci.*
- Jin, M., Liu, K., Bowden, R.O., 2007. A two-stage algorithm with valid inequalities for the split delivery vehicle routing problem. *Int. J. Prod. Econ.* 105 (1), 228–242.

- Lahyani, R., Coelho, L.C., Khemakhem, M., Laporte, G., Semet, F., 2015. A multi-compartment vehicle routing problem arising in the collection of olive oil in tunisia. *Omega* 51, 1–10.
- Lee, C.-G., Epelman, M.A., White III, C.C., Bozer, Y.A., 2006. A shortest path approach to the multiple-vehicle routing problem with split pick-ups. *Transp. Res. Part B* 40 (4), 265–284.
- Lihua, S., 2012. Study on the logistics optimization informationization of oil products in petrochemical enterprises. *Comput. Appl. Chem.* 29 (5), 620–624.
- Luo, Z., Qin, H., Zhu, W., Lim, A., 2016. Branch and price and cut for the split-delivery vehicle routing problem with time windows and linear weight-related cost. *Transp. Sci.* 51 (2), 668–687.
- Melechovský, J., 2013. A variable neighborhood search for the selective multi-compartment vehicle routing problem with time windows. *Lect. Note. Manag. Sci.* 5, 159–166.
- Mendoza, J.E., Castanier, B., Guéret, C., Medaglia, A.L., Velasco, N., 2010. A memetic algorithm for the multi-compartment vehicle routing problem with stochastic demands. *Comput. Oper. Res.* 37 (11), 1886–1898.
- Mendoza, J.E., Castanier, B., Guéret, C., Medaglia, A.L., Velasco, N., 2011. Constructive heuristics for the multicompartiment vehicle routing problem with stochastic demands. *Transp. Sci.* 45 (3), 346–363.
- Muyldermans, L., Pang, G., 2010. On the benefits of co-collection: experiments with a multi-compartment vehicle routing algorithm. *Eur. J. Oper. Res.* 206 (1), 93–103.
- Ng, W.L., Leung, S., Lam, J., Pan, S., 2008. Petrol delivery tanker assignment and routing: a case study in hong kong. *J. Oper. Res. Soc.* 59 (9), 1191–1200.
- Oppen, J., Løkketangen, A., 2008. A tabu search approach for the livestock collection problem. *Comput. Oper. Res.* 35 (10), 3213–3229.
- Pisinger, D., Ropke, S., 2007. A general heuristic for vehicle routing problems. *Comput. Oper. Res.* 34 (8), 2403–2435.
- Popović, D., Vidović, M., Radivojević, G., 2012. Variable neighborhood search heuristic for the inventory routing problem in fuel delivery. *Expert Syst. Appl.* 39 (18), 13390–13398.
- Ropke, S., Pisinger, D., 2006. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transp. Sci.* 40 (4), 455–472.
- Shaw, P., 1998. Using constraint programming and local search methods to solve vehicle routing problems. In: *International conference on principles and practice of constraint programming*. Springer, pp. 417–431.
- Sierksma, G., Tijssen, G.A., 1998. Routing helicopters for crew exchanges on off-shore locations. *Ann. Oper. Res.* 76, 261–286.
- Vidović, M., Popović, D., Ratković, B., 2014. Mixed integer and heuristics model for the inventory routing problem in fuel delivery. *Int. J. Prod. Econ.* 147, 593–604.